

Gestion des IAM pour projet NappeCast

Vue d'ensemble de tous les rôles, utilisateurs et politiques IAM actuellement en place pour les trois services déployés (Airflow, MLflow, app_api), à partir des workflows CI/CD et scripts de déploiement actuels.

DEUX CLÉS D'ACCÈS AWS STATIQUES EXPOSÉES ET PARTAGÉES

`docker/mlflow/aws_create_secret.sh` et `docker/app_api/aws_create_secret.sh` contiennent tous les deux, en clair, la même paire `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`. Une clé statique unique partagée entre deux services va à l'encontre du principe d'isolation appliqué partout ailleurs (chaque service a son propre rôle, secret, sans accès croisés) et une clé compromise sur un service compromet l'autre.

Actions immédiates : désactiver puis supprimer cette clé dans IAM à l'Users à Security credentials ; si le script est commité dans Git, traiter la clé comme définitivement compromise ; la remplacer par un rôle IAM attaché à chaque instance EC2 (étapes 3 et 4 ci-dessous). Le mot de passe RDS présent dans `appapi/aws_create_secret.sh` mérite la même rotation par prudence.

- 1 Vue d'ensemble
- 2 Rôle OIDC CI/CD
- 3 Rôles EC2
- 4 Remplacer les clés statiques
- 5 Secrets Manager
- 6 Checklist

1 Vue d'ensemble des identités IAM

Deux catégories d'identités interviennent dans ce compte : celle qui **déclenche** les déploiements (GitHub Actions, via OIDC) et celles qui **font tourner** les services (rôles d'instance EC2). Aucune des deux ne devrait jamais avoir besoin d'une clé d'accès AWS classique et c'est justement le point que les deux scripts `aws_create_secret.sh` ci-dessus enfreignent.

IDENTITÉ	TYPE	UTILISÉE PAR	PORTÉE ACTUELLE
<code>github-actions-deploy</code>	Rôle IAM, assumé via OIDC	Les 3 workflows CI/CD (Airflow, MLflow, app_api)	ECR (4 repos), <code>ec2:Authorize/RevokeSecurityGroupIngress</code> (3 security groups)
Rôle d'instance Airflow	Rôle IAM EC2 (instance profile)	L'EC2 Airflow (<code>deploy.sh</code> , sidecar S3, conteneurs)	Secret <code>airflow/prod-latest</code> , ECR <code>airflow</code> , S3 <code>nappecast</code> (préfixes DAGs/data/logs)
Rôle d'instance MLflow	Rôle IAM EC2 à créer	L'EC2 MLflow	Aujourd'hui : rien à remplacer par une clé statique. Devrait couvrir : secret <code>mlflow/prod-latest</code> , ECR <code>mlflow</code> , S3 <code>nappecast/mlflow-artifacts/*</code>
Rôle d'instance app_api	Rôle IAM EC2 à créer	L'EC2 app_api (caddy, streamlit, api)	Aujourd'hui : rien à remplacer par une clé statique. Devrait couvrir : secret <code>appapi/prod-latest</code> , ECR <code>api</code> et <code>streamlit</code> , S3 (préfixes utilisés par l'API pour lire/écrire les CSV traités)
Utilisateur IAM à clé statique	Utilisateur IAM	MLflow et app_api (clé partagée)	Portée réelle inconnue tant que la policy de cet utilisateur n'a pas été relue dans la console à faire avant de la supprimer, pour vérifier qu'aucun autre usage ne dépend d'elle.

2 R  le OIDC   " github-actions-deploy

Les trois workflows (`cd-airflow.yml` , `cd-mlflow.yml` , `ci-app_api.yml`) assument tous le m  me r  le : `arn:aws:iam::844099234486:role/github-actions-deploy` . C'est un choix d'architecture diff  rent d'un r  le par service   "    assumer consciemment : une compromission de ce r  le (ou de sa trust policy) impacte les quatre pipelines de build    la fois. Le compromis inverse (un r  le par service) demande de dupliquer la trust policy OIDC trois fois ; les deux approches sont d  fendables, mais la mutualisation actuelle m  rite d'  tre un choix document   plut  t qu'un hasard d'impl  mentation.

Ce r  le a besoin, au minimum, des droits suivants   " vus fonctionner dans les runs r  els :

```
JSON   " PERMISSIONS ATTACH  ES    GITHUB-ACTIONS-DEPLOY
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "EcrAuth", "Effect": "Allow", "Action": "ecr:GetAuthorizationToken", "Resource": "*" },
    {
      "Sid": "EcrPushPull",
      "Effect": "Allow",
      "Action": [
        "ecr:DescribeImages",
        "ecr:BatchCheckLayerAvailability",
        "ecr:BatchGetImage",
        "ecr:GetDownloadUrlForLayer",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:PutImage"
      ],
      "Resource": [
        "arn:aws:ecr:eu-west-3:844099234486:repository/airflow",
        "arn:aws:ecr:eu-west-3:844099234486:repository/mlflow",
        "arn:aws:ecr:eu-west-3:844099234486:repository/api",
        "arn:aws:ecr:eu-west-3:844099234486:repository/streamlit"
      ]
    },
    {
      "Sid": "OpenCloseSsh",
      "Effect": "Allow",
      "Action": ["ec2:AuthorizeSecurityGroupIngress", "ec2:RevokeSecurityGroupIngress"],
      "Resource": [
        "arn:aws:ec2:eu-west-3:844099234486:security-group/",
        "arn:aws:ec2:eu-west-3:844099234486:security-group/",
        "arn:aws:ec2:eu-west-3:844099234486:security-group/"
      ]
    }
  ]
}
```

ECR:DESCRIBEIMAGES CONDITIONNE LE CHECK    IMAGE EXISTE D  J   

Les trois workflows appellent `aws ecr describe-images` avant de builder, pour   viter de repousser un tag d  j pr  sent (repos en tag immutability). Sans ce droit exact sur **chacun** des quatre repos, l'appel   choue par permission, l'  chec est aval   (`>/dev/null 2>&1`), et le script conclut    tort que l'image n'existe pas   " le build repart inutilement et casse au push. V  rifiez   galement l'orthographe exacte de chaque

nom de repository dans la policy : une coquille (`airflow` au lieu de `airflow` , par exemple) donne exactement ce symptôme et est invisible à l'œil sur une policy dense.

Ce rôle n'a **jamais** besoin d'accès à Secrets Manager ni à S3 : les fichiers `docker-compose.yml` / `deploy.sh` voyagent directement par SCP, et c'est chaque instance qui va chercher son propre secret avec son propre rôle (étape 3).

3 Rôles d'instance EC2 à un par service

Chaque `deploy.sh` appelle `aws secretsmanager get-secret-value` et `aws ecr get-login-password` directement depuis l'instance, sans clé fournie en argument : ces appels ne fonctionnent que si un rôle IAM est attaché à l'instance (instance profile). C'est le cas pour Airflow ; il manque pour MLflow et `app_api` (remplacé aujourd'hui par la clé statique partagée).

JSON du rôle d'instance MLFLOW (à créer)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadOwnSecret",
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": "arn:aws:secretsmanager:eu-west-3:844099234486:secret:mlflow/prod-latest-*"
    },
    {
      "Sid": "EcrAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "EcrPull",
      "Effect": "Allow",
      "Action": ["ecr:BatchGetImage", "ecr:GetDownloadUrlForLayer"],
      "Resource": "arn:aws:ecr:eu-west-3:844099234486:repository/mlflow"
    },
    {
      "Sid": "ArtifactStore",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::nappecast",
        "arn:aws:s3:::nappecast/mlflow-artifacts/*"
      ]
    }
  ]
}
```

Même schéma pour `app_api`, en adaptant le secret (`appapi/prod-latest`), les repos ECR (`api` et `streamlit`) et le ou les préfixes S3 résultamment lus/écrits par le code de `src/api` (à confirmer à côté du code : les DAGs suggèrent au moins un accès en lecture des CSV « processed » et en écriture aux logs archivés).

RATTACHER UN RÔLE À UNE INSTANCE DÉJÀ LANCÉE

Pas besoin de relancer l'instance : IAM ajoute le rôle avec une trust policy `ec2.amazonaws.com` , puis EC2 ajoute l'instance à l'instance à l'action `Actions à Security à Modify IAM role`. Prend effet immédiatement, aucun redémarrage requis.

4 Retirer la clé statique partagée

Une fois les deux rôles d'instance (étape 3) attachés et testés (un `docker` compose up réussi sur chaque instance sans la clé statique dans l'environnement), la suppression se fait dans cet ordre :

1. Retirer `AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` des deux secrets Secrets Manager (`mlflow/prod-latest` , `appapi/prod-latest`) et des deux `aws_create_secret.sh` versionnés dans Git.
2. Retirer la ligne `environment: [AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY]` du service `api` dans `docker/app_api/docker-compose.yml` (voir le document `app_api` à ce bloc a de toute façon un autre bug qui le rend inopérant).
3. Vérifier avec `docker exec <conteneur> env | grep AWS` qu'aucune variable de clé statique ne traîne encore dans les conteneurs après redéploiement.
4. Dans IAM à Users, désactiver puis supprimer la clé d'accès (pas l'utilisateur en entier tant que vous n'avez pas confirmé qu'il ne sert à rien d'autre).

5 Secrets Manager à un secret par service, aucun accès croisés

SECRET	CONTENU (CLÉS, PAS VALEURS)	LU PAR
<code>airflow/prod-latest</code>	<code>AIRFLOW__CORE__FERNET_KEY</code> , <code>AIRFLOW__DATABASE__SQL_ALCHEMY_CONN</code> , <code>AIRFLOW__CELERY__RESULT_BACKEND</code> , <code>AIRFLOW__CELERY__BROKER_URL</code> , <code>POSTGRES_PASSWORD</code> , <code>_AIRFLOW_WWW_USER_USERNAME</code> , <code>_AIRFLOW_WWW_USER_PASSWORD</code>	Rôle d'instance Airflow uniquement
<code>mlflow/prod-latest</code>	Aujourd'hui : la clé AWS statique (à retirer, étape 4). Une fois le rôle d'instance en place, ce secret peut redevenir vide ou héberger une future valeur réellement sensible (ex. un secret d'auth si MLflow en gagne un).	Rôle d'instance MLflow uniquement
<code>appapi/prod-latest</code>	<code>PIPELINE_SECRET</code> , la clé AWS statique (à retirer), <code>NAPPECAST_BACKEND_STORE_URI</code> (mot de passe RDS inclus)	Rôle d'instance app_api uniquement

LE NOM DE BASE DOIT RESTER COHÉRENT PARTOUT OÙ IL APPARAÎT

Comme pour Airflow (`POSTGRES_DB: airflow` aligné avec le `/airflow` des URI), le nom de base `nappecast_data` dans `NAPPECAST_BACKEND_STORE_URI` doit rester identique partout où l'app y fait référence afin qu'une divergence ne se voit qu'au premier appel réel à la base, jamais au déploiement.

6 Checklist

- ✓ **Clé AWS statique partagée désactivée et supprimée**, remplacée par un rôle d'instance sur MLflow et app_api.
- ✓ **Mot de passe RDS tourné** une fois l'accès applicatif validé avec le nouveau mécanisme.
- ✓ **Chaque rôle d'instance scope à son propre secret** (`Resource` avec l'ARN exact, jamais `secret:*`) et à son propre repo ECR.

✓ `ecr:DescribeImages` présent sur les quatre repos dans la policy de `github-actions-deploy`.

✓ **Aucun** `aws_create_secret.sh` ne contient plus de valeur réelle” remplacer les vraies clés/mots de passe par des placeholders avant tout commit futur, la valeur réelle se pose une seule fois à la main ou via `put-secret-value` hors d’p’t.

✓ **Décision documentée** sur le partage du rôle CI (`github-actions-deploy`) entre les trois pipelines” accepté tel quel, ou séparé en trois rôles si le blast radius partagé n'est pas souhaité.

Voir aussi « Pipeline Secrets AWS » pour le mécanisme générique (OIDC, `deploy.sh`, User Data), et les documents d’Airflow / MLflow / app_api pour le détail service par service.